

Python Functions Coding Assignment – Question Paper

Python Coding Assignment

Topic: Python Functions

Submission format: Submit one `.ipynb` file.

Total Questions: 30

Instructions

1. Write clean and readable Python code.
2. The first cell of the jupyter notebook should contain your details like name, email, github username.
3. Use meaningful function names exactly as mentioned in the questions wherever given.
4. Test every function with at least two different examples.
5. Do not copy-paste solutions from the internet.
6. Add comments wherever required to explain important logic.
7. Submit only your final Jupyter Notebook file ensuring the file name should be your full name like `<your_full_name>.ipynb` .

Q1. Write a function `welcome_student(name)` that takes a student name and prints the following message:

```
Welcome <name> to Python Functions Class
```

Q2. Write a function `add_two_numbers(a, b)` that returns the sum of two numbers.

Example:

```
add_two_numbers(10, 20)
```

Expected output:

30

Q3. Write a function `calculate_area(length, width)` that returns the area of a rectangle.

Q4. Write a function `is_even(number)` that returns `True` if the number is even, otherwise returns `False`.

Q5. Write a function `student_result(marks)` that returns `"Pass"` if marks are greater than or equal to 40, otherwise returns `"Fail"`.

Q6. Write two functions named `print_square(number)` and `return_square(number)`. The first function should display the square of the number. The second function should return the square of the number. Call both functions and observe the difference.

Q7. Write a function `greet()` that only prints `"Hello"` and does not return anything. Store the function call in a variable and print that variable. Add a short comment explaining the output.

Q8. Write a function `get_student_grade(marks)` that returns the grade according to the conditions below:

```
A if marks >= 90
B if marks >= 75
C if marks >= 60
D if marks >= 40
Fail otherwise
```

Q9. Write a function `total_marks(*marks)` that accepts any number of subject marks and returns the total.

Example:

```
total_marks(80, 70, 90, 60)
```

Expected output:

300

Q10. Write a function `average_marks(*marks)` that accepts any number of marks and returns the average. If no marks are passed, return `"Please provide at least"`

one mark" .

Q11. Write a function `highest_number(*numbers)` that returns the largest number from the given numbers.

Q12. Write a function `multiply_all(*numbers)` that multiplies all the given numbers.

Example:

```
multiply_all(2, 3, 4)
```

Expected output:

```
24
```

Q13. Write a function `department_salary_average(*salaries)` that takes average salaries of different departments and returns the overall average salary.

Q14. Write a function `count_even_odd(*numbers)` that returns how many even and odd numbers are passed.

Example:

```
count_even_odd(1, 2, 3, 4, 5, 6)
```

Expected output:

```
Even: 3, Odd: 3
```

Q15. Write a function `student_info(**details)` that prints all student details in key-value format.

Example:

```
student_info(name="Amit", roll_no=101, course="AI")
```

Expected output:

```
name: Amit  
roll_no: 101  
course: AI
```

Q16. Write a function `employee_profile(**profile)` that returns a formatted string containing employee name, department, and salary.

Example:

```
employee_profile(name="Ravi", department="IT", salary=5000
0)
```

Q17. Write a function `create_bill(**items)` where keys are item names and values are prices. Return the total bill amount.

Example:

```
create_bill(milk=50, bread=40, butter=120)
```

Expected output:

```
210
```

Q18. Write a function `find_student_marks(student_name, **students)` that takes student marks using keyword arguments and returns the marks of the requested student.

Example:

```
find_student_marks("Rahul", Rahul=85, Priya=92, Aman=76)
```

Expected output:

```
85
```

Q19. Write a function `divide(numerator, denominator, /)` where both values must be passed positionally. Test the following calls and write the output or error in a comment:

```
divide(10, 2)
divide(numerator=10, denominator=2)
```

Q20. Write a function using the structure below:

```
def ratio(numerator, denominator, /, *, as_percent=False):
```

The function should return the normal ratio when `as_percent=False` and percentage when `as_percent=True`.

Example:

```
ratio(1, 4)
```

Expected output:

```
0.25
```

Example:

```
ratio(1, 4, as_percent=True)
```

Expected output:

```
25.0
```

Q21. Write a function `student_fee(name, /, *, course, fee)` where `name` must be passed positionally and `course`, `fee` must be passed with their names.

Q22. Write a function `apply_discount(price, discount=10)` that applies a default discount of 10 percent.

Example:

```
apply_discount(1000)
```

Expected output:

```
900
```

Q23. Write a function `calculate_final_price(price, tax_rate=0.05, discount=0)` that subtracts discount first and then applies tax.

Q24. Write a function `course_registration(name, course="Python")` that returns the following message:

```
<name> registered for <course>
```

Test it with and without passing the course name.

Q25. Create a function `append_bad(value, bucket=[])` and call it multiple times. Observe how the list keeps previous values. Then write a corrected version named `append_good(value, bucket=None)`. Add comments explaining the difference.

Q26. Write a function `add_student_bad(student, student_list=[])` that shows the same issue with repeated function calls. Then fix it using `None`.

Q27. Write the function below:

```
def add(a, b, c):  
    return a + b + c
```

Create a list:

```
numbers = [10, 20, 30]
```

Call the function using the list values without manually writing each value separately.

Q28. Write the function below:

```
def create_user(name, city, roll_no):
```

Create a dictionary:

```
user_data = {  
    "name": "Pradeep",  
    "city": "Noida",  
    "roll_no": 101  
}
```

Call the function using the dictionary values without manually writing each value separately.

Q29. Write a function with the following structure:

```
def average(*numbers: float) -> float:
```

The function should return the average of all numbers. If no number is passed, return `0.0`.

Q30. Write a function with the following structure:

```
def format_student(name: str, roll_no: int, marks: float) -  
> str:
```

It should return a formatted student summary.

Example output:

```
Student Rahul with roll number 101 scored 89.5 marks
```

End of Question Paper